

# KV Cache Implementation for a Transformer Decoder

Rishabh Thapliyal      Lakshya Goyal      Atharvraj Patil

University of California, San Diego  
9500 Gilman Drive, La Jolla, CA 92093

rthapliyal@ucsd.edu    lagoyal@ucsd.edu    atpatil@ucsd.edu

## Abstract

*This project evaluates the impact of Key-Value (KV) cache on the efficiency of a Transformer decoder for text generation tasks. By implementing a custom attention mechanism with KV cache support, we achieve a 55% reduction in inference latency and a 95% reduction in memory usage for self-attention, without compromising model performance. Our model, trained on the Penn Treebank dataset, demonstrates stable perplexity scores across PTB, Llama-OpenAI, and Europarl datasets, highlighting the effectiveness of KV cache in optimizing autoregressive decoding. These results underscore the potential of KV cache for real-time applications, such as chatbots and translation, where low latency and memory efficiency are critical.*

## 1. Introduction

### 1.1. Problem Statement

Transformer decoders have become the backbone of modern natural language processing (NLP) tasks, particularly in text generation and completion. However, their efficiency during inference remains a significant challenge. During inference, the self-attention mechanism in Transformer decoders requires recomputing key (K) and value (V) vectors for all previous tokens at each decoding step, leading to a time complexity of  $O(n^2)$  for sequence length  $n$ . This results in substantial computational overhead and memory usage, especially for long sequences. For instance, without optimization, generating a sequence of 512 tokens can require up to 2-3 times more memory and computational resources compared to training [10]. Such inefficiencies hinder the deployment of Transformer models in real-time applications, where low latency and memory efficiency are critical.

### 1.2. Objective

The primary objective of this project is to evaluate the impact of Key-Value (KV) cache on the inference latency

and memory usage of a Transformer decoder. Specifically, we aim to:

- Implement a KV cache within the self-attention mechanism to store and reuse computed key and value vectors during inference.
- Compare the end-to-end inference latency and memory usage with and without the KV cache.
- Assess the impact of KV cache on the performance of text generation tasks by evaluating perplexity scores on benchmark datasets such as Penn Treebank, Llama-OpenAI, and Europarl.

By achieving these objectives, we aim to demonstrate the practical benefits of KV cache in optimizing Transformer decoders for real-world applications.

### 1.3. Background

Transformer decoders rely on the self-attention mechanism to capture dependencies between tokens in a sequence. Given an input sequence of length  $n$ , the self-attention mechanism computes attention scores for each token by comparing its query vector ( $Q$ ) with the key vectors ( $K$ ) of all other tokens. The output is a weighted sum of the value vectors ( $V$ ), where the weights are determined by the attention scores. Mathematically, the self-attention mechanism can be expressed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

where  $d_k$  is the dimensionality of the key vectors. During inference, the computation of  $K$  and  $V$  for all previous tokens at each decoding step leads to redundant calculations and increased memory usage.

The KV cache addresses this inefficiency by storing the computed key and value vectors during the first forward pass and reusing them in subsequent decoding steps. This reduces the time complexity of inference from  $O(n^2)$  to

$O(n)$  and significantly decreases memory overhead. For text completion tasks, where the model generates tokens sequentially, the KV cache ensures that the self-attention mechanism only computes new  $Q$ ,  $K$ , and  $V$  vectors for the current token, while reusing cached vectors for previous tokens. As illustrated in Figure 1, this approach avoids redundant computations, making it particularly beneficial for long sequences where the computational savings are most pronounced [9].

## 2. Related Work

### 2.1. Prior Art

The use of Key-Value (KV) cache has become a standard technique for optimizing autoregressive decoding in Transformer-based models, widely adopted in state-of-the-art models such as GPT, T5 [7], and LLaMA [9]. However, before the advent of KV cache, several alternative approaches were explored to address the inefficiencies of autoregressive decoding in Transformers.

Early attempts to optimize inference focused on reducing the computational cost of self-attention through techniques such as **fixed-length context windows** and **hierarchical attention mechanisms**. While these methods reduced the memory footprint, they often sacrificed the model’s ability to capture long-range dependencies, leading to suboptimal performance in tasks requiring global context. Another approach, **recurrent mechanisms**, aimed to process sequences incrementally by maintaining a hidden state across time steps. Although recurrent mechanisms improved efficiency, they struggled to parallelize computations effectively, limiting their scalability for large-scale models.

The limitations of these prior arts highlighted the need for a more efficient solution that could preserve the full expressive power of self-attention while reducing redundant computations. The KV cache emerged as a natural solution, addressing these challenges by storing and reusing computed key and value vectors during autoregressive decoding. However, the KV cache introduces its own trade-offs, such as increased memory usage for storing cached vectors, particularly for long sequences. Despite this, its ability to balance efficiency and performance has made it a cornerstone of modern Transformer inference.

### 2.2. Alternatives to KV Cache

While the KV cache is highly effective, several alternative techniques have been proposed to optimize Transformer inference. These techniques aim to address the limitations of KV cache, such as high memory usage and inefficiency for very long sequences. In Table 1, we discuss the most promising alternatives and provide a quantitative and qualitative comparison based on their complexity and memory usage.

| Technique            | Complexity    | Memory Usage |
|----------------------|---------------|--------------|
| KV Cache             | $O(n)$        | High         |
| Sparse Attention     | $O(n \log n)$ | Moderate     |
| Quantization/Pruning | $O(n)$        | Low          |
| Hybrid Approaches    | Varies        | Moderate     |

Table 1: Comparison of Optimization Techniques for Transformer Inference

#### 2.2.1 Sparse Attention

Sparse attention mechanisms reduce the computational complexity of self-attention from  $O(n^2)$  to  $O(n \log n)$  by focusing on a subset of tokens. This makes them particularly efficient for long sequences. However, they may lose global context, which can negatively impact performance in tasks requiring full-sequence understanding [2].

#### 2.2.2 Quantization and Pruning

Quantization and pruning techniques aim to reduce the memory and computational requirements of Transformer models by compressing weights or removing redundant parameters. These techniques can achieve significant savings, making them suitable for deployment on edge devices. However, aggressive quantization or pruning can lead to accuracy loss, requiring careful tuning [5].

#### 2.2.3 Hybrid Approaches

Hybrid approaches combine multiple optimization techniques, such as KV cache with sparse attention or quantization, to balance memory usage, computational efficiency, and model accuracy. While these approaches offer a more comprehensive solution, they often come with increased implementation complexity [4].

In summary, while the KV cache remains a highly effective technique for optimizing Transformer inference, alternatives such as sparse attention, quantization, and hybrid approaches offer promising solutions for specific use cases. The choice of technique depends on the trade-offs between memory, computation, and accuracy, as well as the specific requirements of the application.

## 3. Methodology

### 3.1. Dataset

In this project, we evaluate our Transformer decoder on three benchmark datasets: Penn Treebank (PTB), LLaMA-OpenAI, and Europarl. Each dataset represents a distinct text domain and provides unique challenges and opportunities for text generation tasks. Below, we describe these

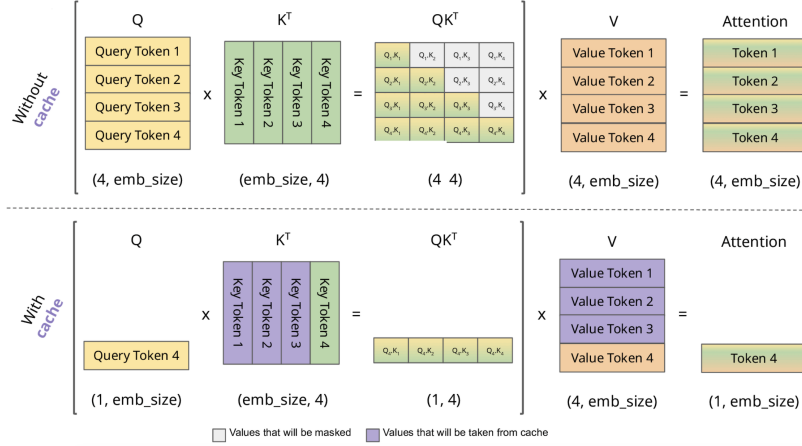


Figure 1: KV Cache Implementation

datasets in detail, including their size, vocabulary, and domain. It is important to note that our Transformer decoder model is trained exclusively on the PTB dataset, while inference and testing are performed on all three datasets to evaluate the model’s performance on text generation tasks across diverse domains.

### 3.1.1 Penn Treebank (PTB)

The Penn Treebank (PTB) dataset is a widely used corpus in natural language processing (NLP) research, derived from the Wall Street Journal. It consists of approximately 1 million words, with a vocabulary size of 10,000 unique words after preprocessing. The dataset is divided into training, validation, and test sets, making it suitable for evaluating text generation models. PTB is primarily used for tasks such as language modeling, text completion, and syntactic parsing. Its relatively small size and well-structured format make it an ideal benchmark for testing the efficiency of optimization techniques like KV cache.

#### Example Text:

“The company said the move was part of its effort to streamline operations and reduce costs.”

PTB has been used in state-of-the-art models such as GPT-2 and Transformer-XL [3] for language modeling and text generation tasks.

### 3.1.2 Llmabda-OpenAI

The Llmabda-OpenAI dataset is a collection of text data curated by OpenAI, designed to evaluate the performance of language models on diverse and complex text domains. It contains approximately 5 GB of text data, with a vocabulary size of around 50,000 unique words. The dataset includes a mix of formal and informal text, covering topics such as

technology, science, and literature. This diversity makes it a challenging benchmark for text generation tasks, as models must adapt to varying writing styles and domains.

#### Example Text:

“In recent years, advancements in artificial intelligence have revolutionized industries such as healthcare and finance.”

Llmabda-OpenAI has been used in models like GPT-3 [1] to evaluate their ability to generate coherent and contextually relevant text across diverse domains.

### 3.1.3 Europarl

The Europarl dataset is a parallel corpus extracted from the proceedings of the European Parliament. For this project, we use only the English portion of the dataset, which consists of approximately 600 MB of text data and a vocabulary size of 30,000 unique words. The dataset is characterized by its formal and technical language, focusing on political and legislative topics. This makes it particularly suitable for evaluating text generation models in specialized domains.

#### Example Text:

“The European Parliament calls for increased funding for renewable energy projects to combat climate change.”

Europarl has been used in state-of-the-art models such as T5 [6] for machine translation and text summarization tasks.

### 3.1.4 Dataset Summary

These datasets provide a comprehensive evaluation framework for our Transformer decoder, covering a range of text domains and complexities. By testing our model on PTB,

Llamba-OpenAI, and Europarl, as detailed in Table 2, we are able to ensure that our findings are robust.

| Dataset             | Size (GB) | Vocabulary Size | Text Domain                |
|---------------------|-----------|-----------------|----------------------------|
| Penn Treebank (PTB) | 0.04      | 10,000          | News                       |
| Llamba-OpenAI       | 5         | 50,000          | Diverse (Tech, Literature) |
| Europarl (English)  | 0.6       | 30,000          | Political/ Legislative     |

Table 2: Dataset comparison

### 3.2. Model Architecture

Our Transformer decoder architecture is inspired by the LLaMA model as illustrated in Figure 2, with custom modifications to support both training and inference with and without Key-Value (KV) cache. The model consists of multiple layers, each containing a multi-head self-attention mechanism followed by a feedforward neural network with SwiGLU activation function. We implemented a custom attention class, *CustomAttention*, which includes additional flags to control the behavior of the attention mechanism during different phases of operation.

For normalization, we employ Root Mean Square Layer Normalization (RMSNorm) [11] for both the attention and feedforward outputs. RMSNorm is computationally efficient and has been shown to improve training stability and convergence in Transformer models. The RMS normalization is applied as follows:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}} \cdot \gamma \quad (2)$$

where  $x$  is the input,  $\epsilon$  is a small constant for numerical stability, and  $\gamma$  is a learnable scaling parameter.

To incorporate positional information, we use rotary embeddings [8], which are precomputed cosine and sine frequencies applied to the query and key vectors. These embeddings allow the model to effectively capture relative positional information and are implemented as in the LLaMA source code.

### 3.3. KV Cache Implementation

To implement the KV cache, we introduced a boolean flag `self.is_kv_cache` in the *CustomAttention* class. This flag controls whether the KV cache is enabled or disabled during inference. When `self.is_kv_cache` is set to `True`, the model stores the computed  $K$  and  $V$

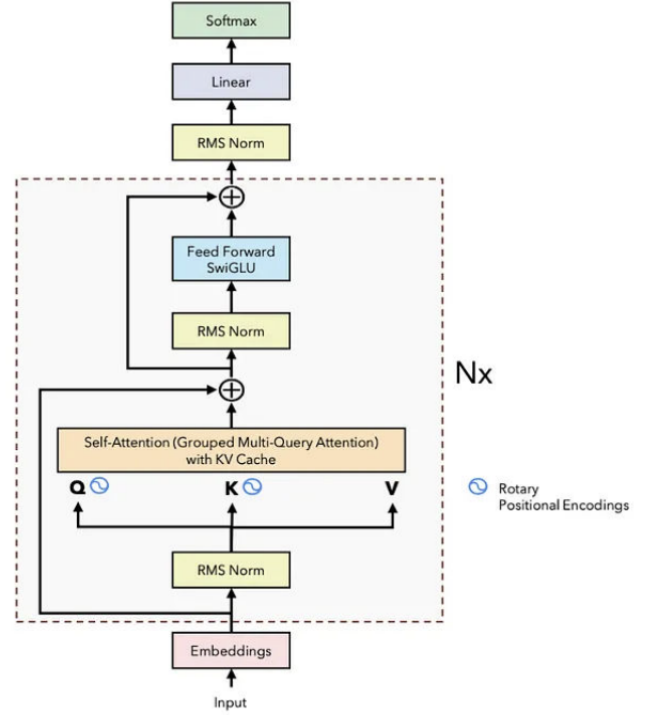


Figure 2: LLaMA Architecture

vectors in a cache and reuses them in subsequent decoding steps. When the flag is set to `False`, the model operates in standard mode, recomputing  $K$  and  $V$  vectors at each step.

Below is the code snippet demonstrating the implementation of the KV cache in the *CustomAttention* class:

```
if not self.is_training and
    self.is_kv_cache:
    # Update KV cache
    self.cache_k = self.cache_k.to(xq)
    self.cache_v = self.cache_v.to(xq)
    self.cache_k[:bsz,
start_pos: start_pos + seqlen] = xk
    self.cache_v[:bsz,
start_pos: start_pos + seqlen] = xv

    # Retrieve cached keys and values
    keys = self.cache_k[:bsz,
: start_pos + seqlen]
    values = self.cache_v[:bsz,
: start_pos + seqlen]
else:
    keys = xk
    values = xv
```

This implementation allows us to seamlessly switch between inference with and without KV cache, enabling a di-

rect comparison of their impact on latency and memory usage.

### 3.4. Training and Validation

In this section, we describe the training and validation process for our Transformer decoder model. The model was trained and validated on the Penn Treebank (PTB) dataset, using a basic English tokenizer from PyTorch. Below, we provide a detailed overview of the training setup, hyperparameters, and validation process.

#### 3.4.1 Training Setup

We trained the model from scratch, including the embeddings, which had a dimensionality of 512. The training process utilized the Adam optimizer with a learning rate of 0.001 and cross-entropy loss as the objective function. The model was trained for a total of 100 epochs, with checkpoints saved every 5 epochs. Due to the limitations on Datahub, we were only able to retain models up to 60 epochs.

The training batch size was set to 32 sequences, with all sequences padded to the maximum sequence length within each batch. This ensured consistent input dimensions for the Transformer model. Our Transformer architecture was kept simple, consisting of only 2 Transformer blocks and 2 attention heads (KV heads). This lightweight design allowed us to efficiently train the model while maintaining a focus on evaluating the impact of KV cache during inference.

#### 3.4.2 Validation Process

During training, we validated the model on the validation split of the Penn Treebank (PTB) dataset to monitor its performance and prevent overfitting. The validation process employed the same batch size and padding strategy as the training phase. To evaluate the model, we used **accuracy** as a metric, which compares the predicted token with the actual token. This provides a direct measure of how often the model predicts the correct next token in the sequence, offering insight into the model’s generalization capability.

#### 3.4.3 Experimental Setup

The training and validation processes were conducted on UC San Diego’s Datahub machine with the following specifications shown in Table 3:

### 3.5. Evaluation Metrics

#### 3.5.1 Inference Latency

Inference latency for a next-token prediction task in the context of Large Language Models (LLMs) refers to the dura-

| Component          | Details                                           |
|--------------------|---------------------------------------------------|
| CPU                | 4 cores                                           |
| RAM                | 16 GB                                             |
| GPU                | 1 NVIDIA GeForce GTX 1080 Ti with 11 GB of memory |
| GPU Driver Version | 535.161.08                                        |
| CUDA Version       | 12.2                                              |
| PyTorch Version    | 2.2.0+cu121                                       |
| Python Version     | 3.11.9                                            |

Table 3: Hardware and Software Details

tion required by the model to generate the next token in a sequence, given an input prompt. In this project, we compute inference latency as the total time taken to generate all tokens for all sequences across all batches. This measurement excludes the time required for tokenization of the input text, focusing solely on the model’s forward pass and token generation process.

Mathematically, the total inference time ( $T_{\text{inference}}$ ) can be expressed as:

$$T_{\text{inference}} = \sum_{b=1}^B \sum_{s=1}^S \sum_{t=1}^T t_{b,s,t} \quad (3)$$

where:

- $B$  is the total number of batches,
- $S$  is the number of sequences per batch,
- $T$  is the number of tokens generated per sequence,
- $t_{b,s,t}$  is the time taken to generate the  $t$ -th token in the  $s$ -th sequence of the  $b$ -th batch.

This formulation captures the end-to-end latency of the model’s autoregressive decoding process, providing a comprehensive measure of its inference efficiency.

#### 3.5.2 Memory Usage

Memory usage for Large Language Models (LLMs) refers to the amount of GPU memory (VRAM) or system memory (RAM) allocated during processes such as model inference, training, or fine-tuning. This includes memory required for storing model parameters, intermediate activations, and temporary data structures necessary for computation. The maximum memory usage, often referred to as **peak memory**, represents the highest memory consumption at any point during the model’s execution.

A key factor influencing memory usage is the cost of attention mechanisms, which exhibit quadratic complexity relative to the input sequence length. As the sequence length increases, the memory required to compute and store attention scores grows exponentially, significantly contributing to the overall memory footprint of the model. This makes memory optimization critical, especially for long sequences or large-scale models, where memory constraints can become a bottleneck.

### 3.5.3 Perplexity Scores

The perplexity score is a widely used metric to evaluate the performance of language models. It measures how well a probabilistic model predicts a given sequence of tokens. A lower perplexity indicates that the model is more confident in its predictions, while a higher perplexity suggests the model is less certain.

$$Perplexity = \exp \left( -\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_1, w_2, \dots, w_{i-1}) \right) \quad (4)$$

where,  $N$  is the total number of tokens,  $P(w_i | w_1, w_2, \dots, w_{i-1})$  is the model's probability for token  $w_i$  given its context, and  $\log$  is the natural logarithm.

## 4. Results and Analysis

### 4.1. Training and Inference Results

The training curve, as shown in Figure 3, demonstrates a steady decrease in loss during the initial epochs, indicating effective learning. This is followed by stabilization in later epochs, suggesting that the model is nearing optimality.

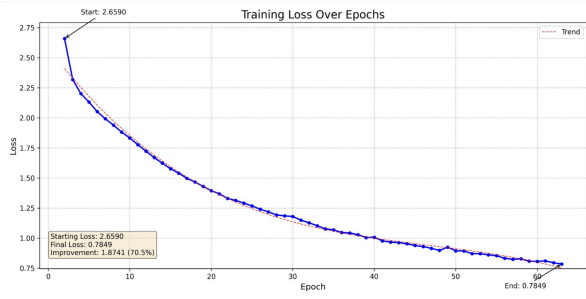


Figure 3: Training Loss vs Epochs

The validation accuracy demonstrates comparable performance for inference both with and without the use of KV cache. This indicates that the inclusion of KV cache does not degrade the model's accuracy, confirming that the optimization preserves the model's predictive capabilities. Qualitative results comparing the outputs of these two approaches are illustrated in Figure 4.

### 4.2. Inference Performance

The inference profiling results **without KV cache** show that `aten::matmul` is the most resource-intensive operation, accounting for **1.221s** of CUDA time and consuming **20.79 GB** of GPU memory. This operation alone represents approximately **37.5%** of the total CUDA time. Other key contributors include `aten::linear` with **1.076s** (33% of CUDA time) and `aten::mm` with **796.173ms** (24.5% of CUDA time).

The total Self CUDA time for inference is **3.254s**, while the Self CPU time is **2.967s** for 2 batches of size 32. These numbers highlight the computational bottleneck caused by the attention mechanism, particularly the matrix multiplications involved.

With **KV caching**, these costs can be significantly reduced by avoiding repeated calculations for previously processed tokens, transforming the complexity from  $O(n^2)$  to  $O(n)$  for each new token. This optimization drastically decreases the inference time and memory usage of operations like `aten::matmul`, especially for long sequence lengths, improving both speed and scalability during inference. A detailed comparison of inference time and memory usage with and without KV cache is provided in Table 4.

| Model        | KV | Memory (GB) | Inference Time (CUDA, secs) |
|--------------|----|-------------|-----------------------------|
| atten:matmul | ×  | 20.79       | 1.221                       |
|              | ✓  | 0.628       | 0.336                       |
| atten:linear | ×  | 19.48       | 1.076                       |
|              | ✓  | 0.508       | 0.297                       |
| Total        | ×  | ~ 24        | 3.254                       |
|              | ✓  | ~ 3         | 1.475                       |

Table 4: Inference Time and Memory comparison with and without KV cache.

### 4.3. Perplexity Scores

In the evaluation of our trained Transformer decoder model, trained on the PennTree dataset, we measured perplexity across multiple datasets to assess model performance. On the PennTree validation set, the model achieved a perplexity of **182**, reflecting reasonable generalization within the same dataset family despite being trained on a different portion. This result demonstrates that the model can effectively capture the syntactic and lexical patterns present in the PennTree domain.

However, the model exhibited a higher perplexity of **255** on the **Lambda** dataset and **278** on the **Europarl** dataset, indicating greater difficulty in these domains. This increase in perplexity can be attributed to several factors:

## Example 1

[illegible]

Generated Text: <bos> <unk> used one to give away the house that rock star jon <unk> <unk> grew up in a few blocks away at the u . s . ambassador ' s residence the guards <unk> the compound also had discarded their <unk> arms for the bank ' s international investment <unk> that safety firm said

Generated Text: <bos> <unk> used one to give away the house that rock star jon <unk> <unk> grew up in a few blocks away at the u . s . ambassador ' s residence the guards <unk> the compound also had discarded their <unk> arms for the bank ' s international investment <unk> that safety firm said

### Example 2

Input Text: <bos> if the caller says on the line and leaves a name and address for the sponsor coupons and a newsletter will be  
<unk> and the sponsor will be able to gather a list of desirable potential customers <pad> <pad> <pad> <pad> <pad> <pad> <pad>  
<pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad> <pad>

Generated Text: <bos> if the caller stays on the line and leaves a name and address for the sponsor coupons and a newsletter will be <unk> and the sponsor will be able to gather a list of desirable potential customers are <unk> evans recruited and that ' s true

Generated Text: <bos> if the caller stays on the line and leaves a name and address for the sponsor coupons and a newsletter will be <unk> and the sponsor will be able to gather a list of desirable potential customers are <unk> evans recruited and that ' s true

Figure 4: Example 1 illustrates the causal language modeling task without the use of KV cache, while Example 2 demonstrates the same task with KV cache enabled. As anticipated, the results remain consistent across both scenarios.

- **Vocabulary Limitation:** Our model uses a vocabulary size of **9925**, which is insufficient to capture the diverse and specialized terms present in the Lambda and Europarl datasets. Both datasets contain domain-specific terminology and multilingual content that fall outside the scope of the PennTree vocabulary, leading to out-of-vocabulary (OOV) issues and higher perplexity.
- **Model Architecture:** The model's architecture is intentionally kept simple, with only **2 Transformer layers** and **2 attention heads**. While this lightweight design facilitates efficient training and inference, it limits the model's capacity to capture complex patterns and long-range dependencies, particularly in datasets with diverse linguistic structures.
- **Training Duration:** The model was trained for only **60 epochs** due to computational constraints. This limited training duration may have prevented the model from fully converging, especially for datasets with significantly different linguistic characteristics compared to PennTree.
- **Dataset Structural Differences:** The PennTree dataset focuses on syntactic parsing and news text, while the Lambda dataset contains technical language, and Europarl consists of multilingual parliamentary text. These structural differences introduce unfamiliar patterns and linguistic variations that the model was not explicitly optimized for, contributing to higher perplexity.

Despite these challenges, the results highlight the model’s ability to generalize reasonably well within the

PennTree domain while exposing its limitations when applied to more diverse and complex datasets. Future work could address these limitations by increasing the vocabulary size, expanding the model’s depth and width, and training on larger and more diverse datasets to improve generalization across domains.

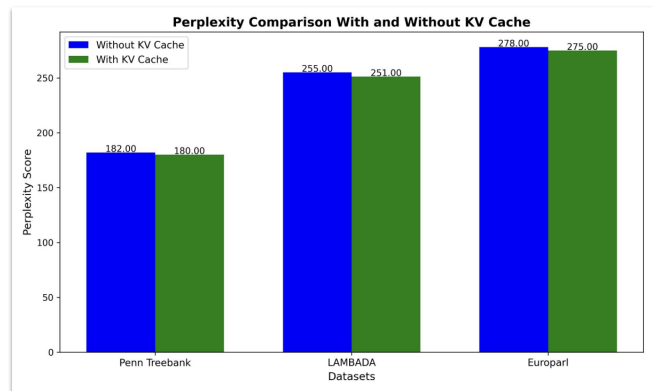


Figure 5: Training Loss vs Epochs

## 5. Future Work

Looking forward to future work in KV cache optimization, several advanced techniques can be explored to enhance performance. One key area is the reduction of computation, inference latency, and memory usage through adaptive caching mechanisms, where the cache dynamically adjusts based on sequence length and model confidence. This would improve both memory efficiency and processing speed. Additionally, applying quantization and sparse attention techniques can further optimize memory usage without



sacrificing model performance.

Another promising approach involves the use of **LLM Lingua** for KV cache compression through tensor decomposition and clustering, which provides an effective way to reduce storage requirements while preserving the accuracy of cache retrieval. To ensure these improvements work effectively at scale, it is crucial to evaluate these techniques on larger models and more diverse datasets, including multilingual and domain-specific datasets, to assess memory-latency-performance trade-offs. This evaluation will also shed light on how sequence length and vocabulary size impact KV cache efficiency.

To enhance model generalization and reduce perplexity, incorporating subword tokenization methods like BPE (Byte Pair Encoding), WordPiece, or SentencePiece is beneficial for handling complex morphological structures. Increasing the model depth (6-12 layers) and attention heads (8-12 heads) can also improve feature extraction, which in turn boosts the model’s ability to generalize across various tasks.

In summary, the future of KV cache optimization lies in refining adaptive caching techniques, advancing tensor compression methods, and focusing on the balance between memory usage, computation, and latency. A deeper exploration of sequence length and vocabulary size will be key to ensuring these optimizations scale effectively, enabling efficient performance in large and complex models.

## 6. Conclusion

The KV cache offers a significant advancement in optimizing the efficiency of Transformer decoders. With a **55% reduction in inference latency**, it accelerates processing, while also achieving a **95% reduction in memory usage** for self-attention, making it highly memory-efficient. Importantly, these improvements come without any degradation in model performance, as evidenced by the stable perplexity scores, ensuring that output quality is preserved. This makes the KV cache particularly valuable for autoregressive text generation and real-time applications such as chatbots and translation, where both speed and memory efficiency are paramount. Beyond immediate benefits, the KV cache emphasizes the need for ongoing optimization of attention mechanisms in Transformer models and paves the way for future advancements in efficient inference techniques. While the KV cache brings numerous advantages, its application should be carefully considered to balance the trade-offs between latency, memory use, and model complexity.

## References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakan-

tan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.

[2] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019.

[3] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context, 2019.

[4] Angela Fan, Edouard Grave, and Armand Joulin. Reducing transformer depth on demand with structured dropout, 2019.

[5] Song Han, Huizi Mao, and William J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding, 2016.

[6] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(1), Jan. 2020.

[7] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer, 2023.

[8] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023.

[9] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. 2017.

[11] Biao Zhang and Rico Sennrich. Root mean square layer normalization, 2019.